

Write Once, Accelerate Everywhere: GPU-Ready Java with TornadoVM

Thanos Stratikopoulos
Research Fellow at the University of Manchester



TORNADOVM



The University of Manchester

February 6th, 2026
Voxxed Days Ticino 2026

About me



Dr Thanos Stratikopoulos
Research Fellow and Impact Champion
The University of Manchester

✉ athanasios.stratikopoulos@manchester.ac.uk

X [@thanos_str](https://twitter.com/thanos_str)

in <https://www.linkedin.com/in/stratika/>

🦋 [@stratika.bsky.social](https://bsky.app/profile/stratika.bsky.social)

Agenda

- Brief History of TornadoVM
- TornadoVM Overview
- TornadoVM Types with Foreign Function & Memory API (Project Panama)
- Setup TornadoVM
- Integration in Java Projects
- More Complex Cases
- Summary

TornadoVM History

How it started...

- PhD research project 2013, Uni. Manchester
- Initially named Jacc it later evolved to TornadoVM
- Originally based on SOOT framework
- Designed to understand how the JVM could tackle GPUs

Boosting Java Performance using GPGPUs

James Clarkson Christos Kotselidis Gavin Brown Mikel Luján

The University of Manchester
(first.last)@manchester.ac.uk

Abstract

Heterogeneous programming has started becoming the norm in order to achieve better performance by running portions of code on the most appropriate hardware resource. Currently, significant engineering efforts are undertaken in order to enable existing programming languages to perform heterogeneous execution mainly on GPUs. In this paper we describe Jacc, an experimental framework which allows developers to program GPGPUs directly from Java. By using the Jacc framework, developers have the ability to add GPGPU support into their applications with minimal code refactoring.

To simplify the development of GPGPU applications we allow developers to model heterogeneous code using two key abstractions: *tasks*, which encapsulate all the information needed to execute code on a GPGPU; and *task graphs*, which capture the inter-task control-flow of the application. Using this information the Jacc runtime is able to automatically handle data movement and synchronization between the host and the GPGPU, eliminating the need for explicitly managing disparate memory spaces.

In order to generate highly parallel GPGPU code, Jacc provides developers with the ability to decorate key aspects of their code using annotations. The compiler, in turn, exploits this information in order to automatically generate code without requiring additional code refactoring.

Finally, we demonstrate the advantages of Jacc, both in terms of programmability and performance, by evaluating it against existing Java frameworks. Experimental results show an average performance speedup of 32x and a 4.4x code decrease across eight evaluated benchmarks on a NVIDIA Tesla K20m GPU.

1. Introduction

Heterogeneous programming languages enable developers to execute portions of their code onto specialized devices. This task, which typically involves offloading work from a host onto a more specialized *device* such as a GPGPU, requires a programming model that supports code execution in different *contexts*. The downside of such programming languages is that they require developers to be aware of the *contexts* when writing code and to ensure that execution and data are synchronized across them. In this paper we demonstrate that a large amount of this responsibility can be eliminated or handled automatically and, consequently, reduce the burden on the developers.

Current established heterogeneous programming languages, such as CUDA [22] and OpenCL [16], require developers to logically separate their applications into code that runs either on the host or on the device (known as a *kernel*). As a consequence, these approaches require additional code to co-ordinate execution between the host and kernels.

In this paper we demonstrate a simplified heterogeneous programming model, in the context of the Java language, through the use of implicit parallelism and data synchronization. The in-

troduced Java Acceleration system (Jacc) shares many similarities with directive-based approaches like OpenAcc [19] and OpenMP 4.0 [20]. However, unlike these approaches it also has the ability to automatically optimize execution by eliminating redundant data transfers and executing kernels out of order. Furthermore, the modular programming interfaces of Jacc enable future extensions that will result in dynamically selecting both the number and types of the underlying devices for code execution (e.g. GPGPUs, FPGAs or ASICs). In order to achieve that, the only information needed from the developer is a model of the heterogeneous portion of the application, captured as a Direct Acyclic Graph (DAG), and the decoration of kernels with metadata that will transparently guide the compilation.

Overall the paper makes the following contributions:

- Introduces an approach to heterogeneous programming that abstracts away low-level hardware details and housekeeping functionalities.
- Presents the design and implementation of Jacc and all its components.
- Showcases the efficiency of Jacc by implementing a GPGPU offloading runtime that is capable of handling the asynchronous nature of heterogeneous programs.
- Provide an in-depth comparative performance analysis of Jacc and standard Java multithreaded benchmarks. The experimental results show that Jacc can provide an order of magnitude improvement in performance while decreasing code size by 4.4x.

2. The Jacc Framework

Jacc is a Java based framework that allows developers to program heterogeneous hardware in a simplified manner. In the context of this paper, we showcase Jacc in programming CUDA enabled GPGPUs directly from Java. As depicted in Figure 1, the two major components of Jacc are: 1) a just-in-time (JIT) compiler to compile Java bytecode into low-level machine code for GPGPUs and 2) a runtime system designed to manage the execution of application kernels on GPGPUs; all the compilation, data movement between devices, synchronization and kernel executions are handled automatically by the runtime.

The design philosophy of Jacc is that by providing the right programming abstractions it is possible to create highly-parallel kernels without forcing developers to change their software engineering practices. To that end, Jacc builds on top of two basic blocks: the *task* and the *task graph*. A task encapsulates all the vital information for executing code in a parallel environment; typically a method reference, a parameter list and some scheduling metadata. Most importantly, tasks are not restricted to execute on specific hardware. They are mapped onto hardware when they are inserted into a task graph, a mapping which can be changed dynamically by the developer. Tasks can be created from any method in the application; however, methods annotated with the `@Jacc` annotation

How it is going...

- Open-sourced in 2018
- Transitioned to Graal
- 23 releases; currently at v2.2.0
- 30 contributors
- Production deployment at
ESA's GAIA mission



UK Research
and Innovation

Funded by the European Union (AERO: 101092850) & UKRI
under the Horizon Europe funding guarantee (AERO:
10048318).

A screenshot of the TornadoVM GitHub repository and website. The top part shows the GitHub interface with the repository name 'TornadoVM', a public status, and various statistics like 12 branches, 34 tags, and 10,086 commits. Below this is a preview of the TornadoVM website, which features a dark theme with a purple circular logo and the text 'Run your software faster and simpler!'. The website also has sections for 'Challenges' and 'Our Solution'. The right side of the screenshot shows the 'About' section of the repository, describing TornadoVM as a practical and efficient heterogeneous programming framework for managed languages, with links to the website and various tags like 'java', 'ai', 'openc1', 'parallel-computing', etc.

<https://aero-project.eu/>

TornadoVM Mature Ecosystem

JDK 21 Distributions

OpenJDK

GraalVM

Eclipse Temurin

RedHat Mandrel

Windows JDK

Azul Zulu

SapMachine

Truffle Framework



Plugin to JDK



TORNADOVM



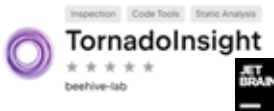
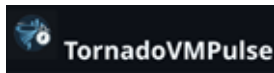
Hardware Backends

OpenJDK25
GraalVM25
(soon)

Docker Images



Developer Tools

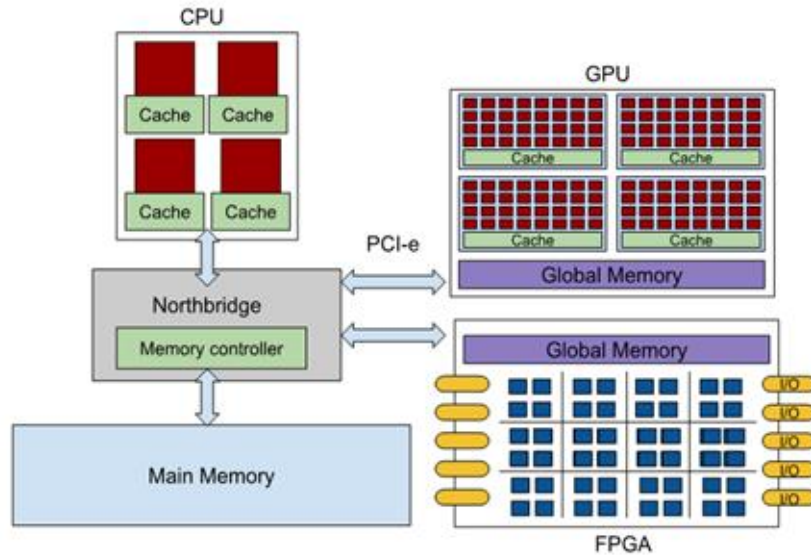


Integrations

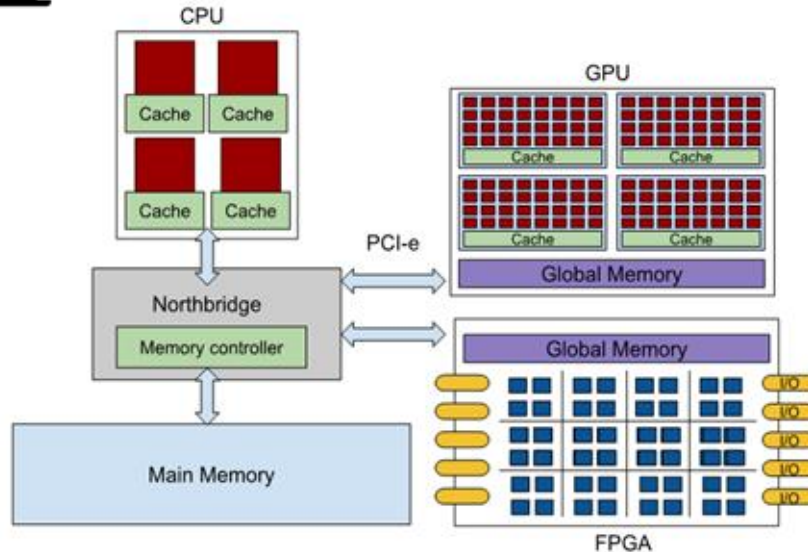


How much different is GPU programming?

Very different!



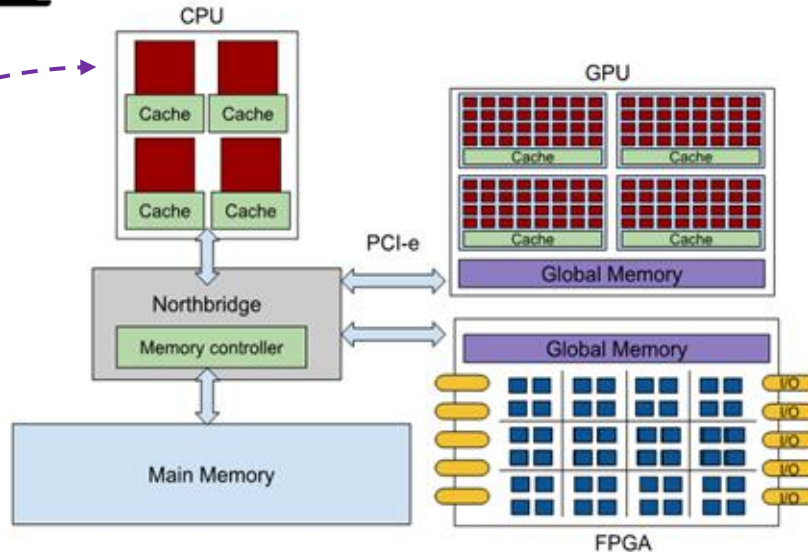
Very different!



Parallel APIs

- Vector API
- Parallel Streams
- Virtual Threads

Very different!



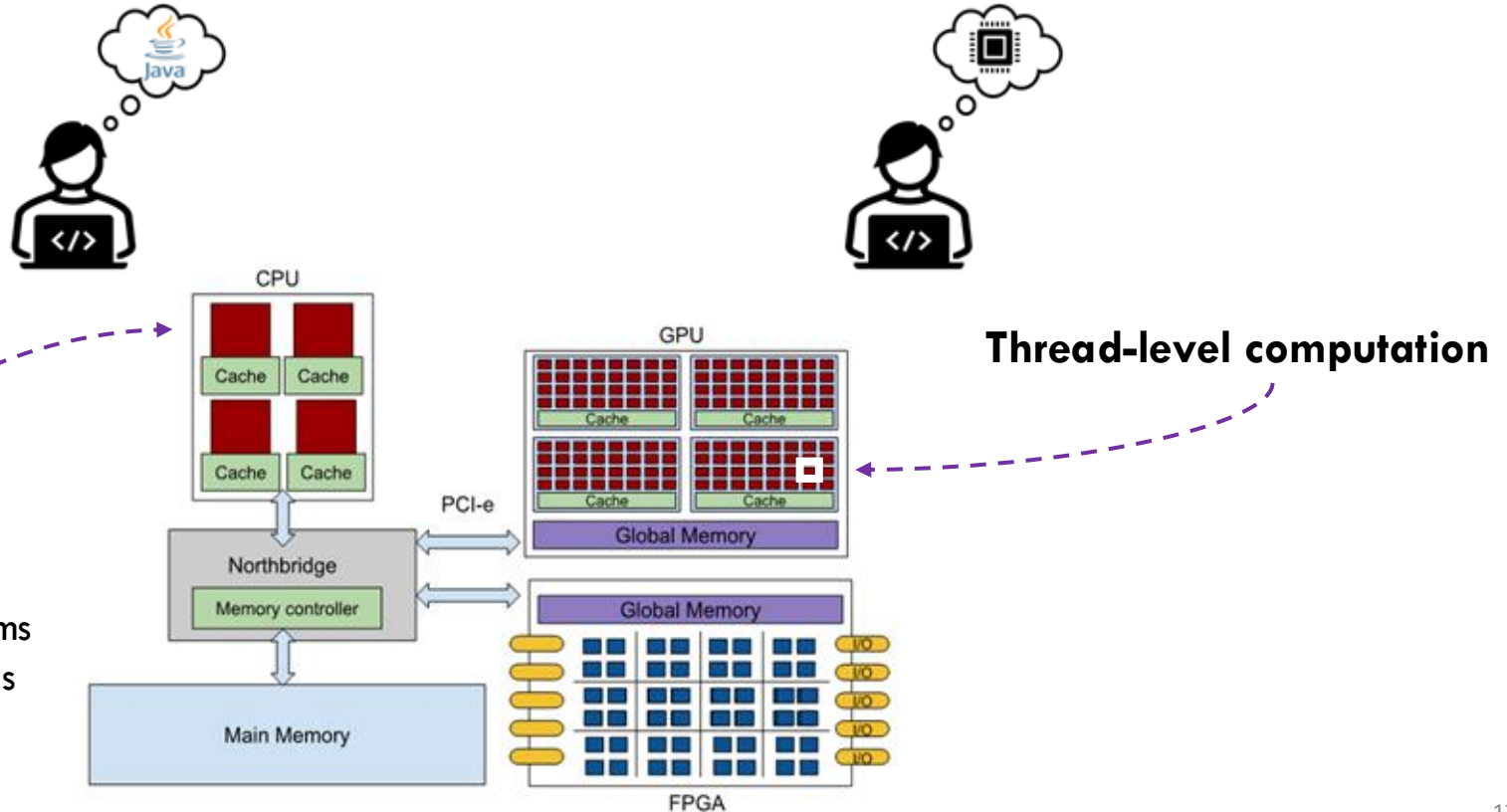
Parallel APIs

- Vector API
- Parallel Streams
- Virtual Threads

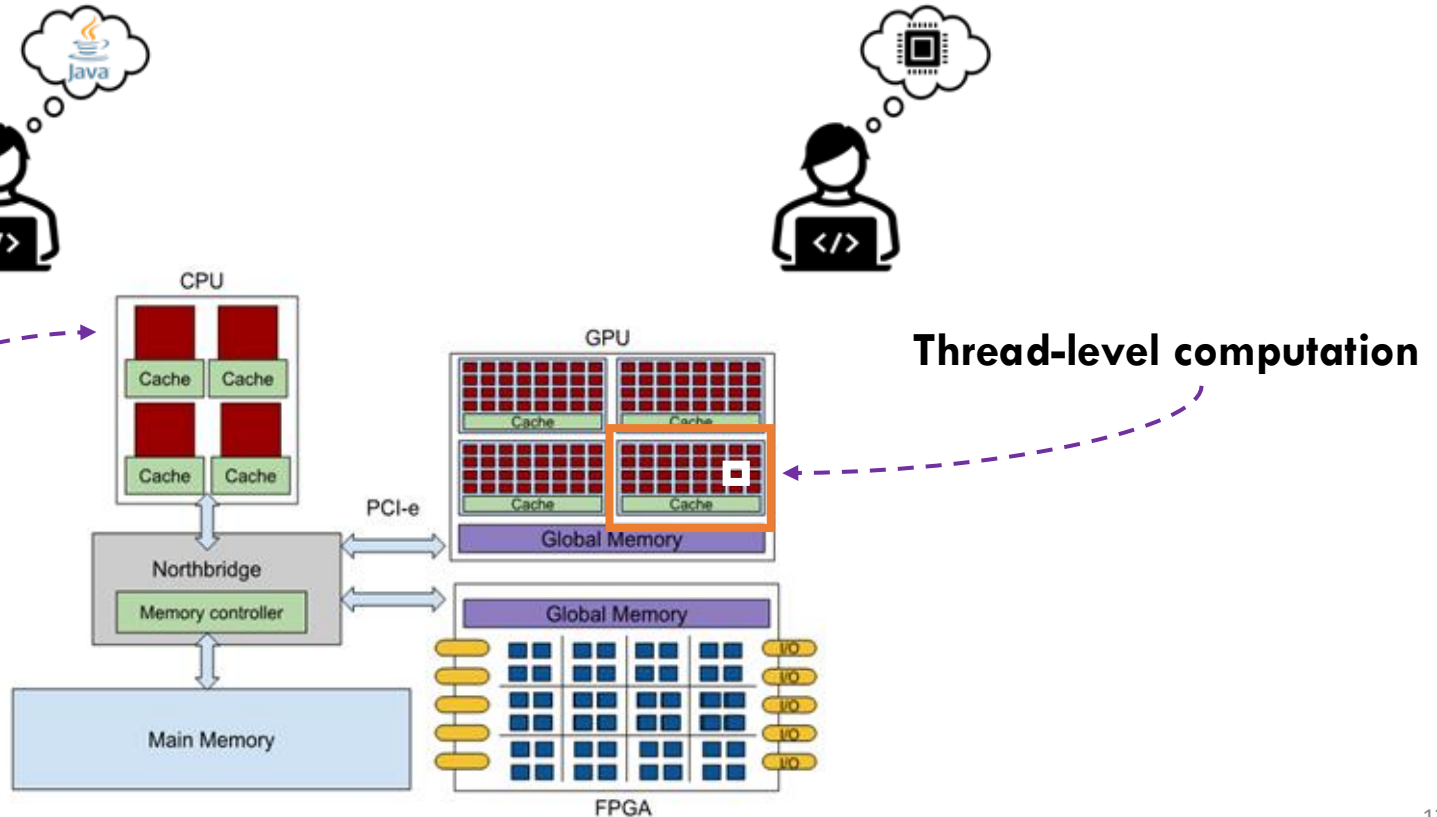
Very different!

Parallel APIs

- Vector API
- Parallel Streams
- Virtual Threads



Very different!



Parallel APIs

- Vector API
- Parallel Streams
- Virtual Threads

Key remarks regarding GPU programming

- GPU programming requires a *different mindset* than programming in Java
- Programmers must be familiar with *thread-level programming* and *grids*
- Lots of *verbosity*, since developers **must** *create explicitly all data structures*



It can break the abstraction! Remember Java's promise:
Write Once, Run Anywhere!

TornadoVM Overview



TORNADOVM



@tornadovm



@tornadovm.org



info@tornadovm.org



<https://tornadovm.org>



<https://github.com/beehive-lab/TornadoVM>



*A JVM plugin that accelerates
Java methods on
heterogeneous hardware!*

Features:

- Open source
- Platform agnostic
- Automatic code optimization
- Off-heap data types
(Foreign Function & Memory API)

TornadoVM Types with Foreign Function & Memory API (Project Panama)

Objects Allocated on Heap

```
public static void main(String[] args) {  
    int size = 512;
```

```
    float[] matrixA = new float[size * size];  
    float[] matrixB = new float[size * size];  
    float[] matrixC = new float[size * size];  
    float[] resultSeq = new float[size * size];
```

Allocated on-heap.
So pruned to GC!

```
    Random r = new Random();  
    IntStream.range(0, size * size).parallel().forEach(idx -> {  
        matrixA[idx] = r.nextFloat();  
        matrixB[idx] = r.nextFloat();  
    });
```

```
    matrixMultiplication(matrixA, matrixB, resultSeq, size);
```

Using Off-heap Data

```
public static void main(String[] args) {
    int size = 512;
```

```
float[] matrixA = new float[size * size];
float[] matrixB = new float[size * size];
float[] matrixC = new float[size * size];
float[] resultSeq = new float[size * size];
```

```
Random r = new Random();
IntStream.range(0, size * size).parallel().forEach(idx -> {
    matrixA[idx] = r.nextFloat();
    matrixB[idx] = r.nextFloat();
});
```

```
matrixMultiplication(matrixA, matrixB, resultSeq, size);
```

```
public static void main(String[] args) throws Exception {
    int size = 512;
```

```
FloatArray matrixA = new FloatArray( numberOfElements: size * size);
FloatArray matrixB = new FloatArray( numberOfElements: size * size);
FloatArray matrixC = new FloatArray( numberOfElements: size * size);
FloatArray resultSeq = new FloatArray( numberOfElements: size * size);
```

```
Random r = new Random();
IntStream.range(0, size * size).parallel().forEach(idx -> {
    matrixA.set(idx, r.nextFloat());
    matrixB.set(idx, r.nextFloat());
});
```

```
matrixMultiplication(matrixA, matrixB, resultSeq, size);
```

<https://tornadovm.readthedocs.io/en/latest/offheap-types.html#migrating-from-v0-15-2-to-v1-0>

Utilizing TornadoVM Off-heap DataTypes

```
// allocate an off-heap memory segment that will contain 16 int values
IntArray intArray = new IntArray(16);
```

Additionally, developers can create an instance of a TornadoVM native array by invoking factory methods for different data representations. In the following examples we will demonstrate the API functions for the `FloatArray` type, but the same methods apply for all support native array types.

```
// from on-heap array to TornadoVM native array
public static FloatArray fromArray(float[] values);
// from individual items to TornadoVM native array
public static FloatArray fromElements(float... values);
// from Memory Segment to TornadoVM native array
public static FloatArray fromSegment(MemorySegment segment);
```

The main methods that the off-heap types expose to manage the Memory Segment of each type are presented in the list below.

```
public void set(int index, float value) // sets a value at a specific index
E.g.:
    FloatArray floatArray = new FloatArray(16);
    floatArray.set(0, 10.0f); // at index 0 the value is 10.0f
public float get(int index) // returns the value of a specific index
E.g.:
    FloatArray floatArray = FloatArray.fromArray(new float[] {2.0f, 1.0f, 2.0f, 5.0f});
    float floatValue = floatArray.get(3); // returns 5.0f
public void clear() // sets the values of the segment to 0
E.g.:
    FloatArray floatArray = new FloatArray(1024);
    floatArray.clear(); // the FloatArray contains 1024 zeros
public void init(float value) // initializes the segment with a specific value
E.g.:
    FloatArray floatArray = new FloatArray(1024);
    floatArray.init(1.0f); // the FloatArray contains 1024 ones
public int getSize() // returns the number of elements in the segment
E.g.:
    FloatArray floatArray = new FloatArray(16);
    int size = floatArray.getSize(); // returns 16
public float[] toHeapArray(); // Converts the data from off-heap to on-heap
public long getNumBytesOfSegment(); // Returns the total number of bytes the underlying Memory
public long getNumBytesWithoutHeader(); // Returns the total number of bytes the underlying Mem
```

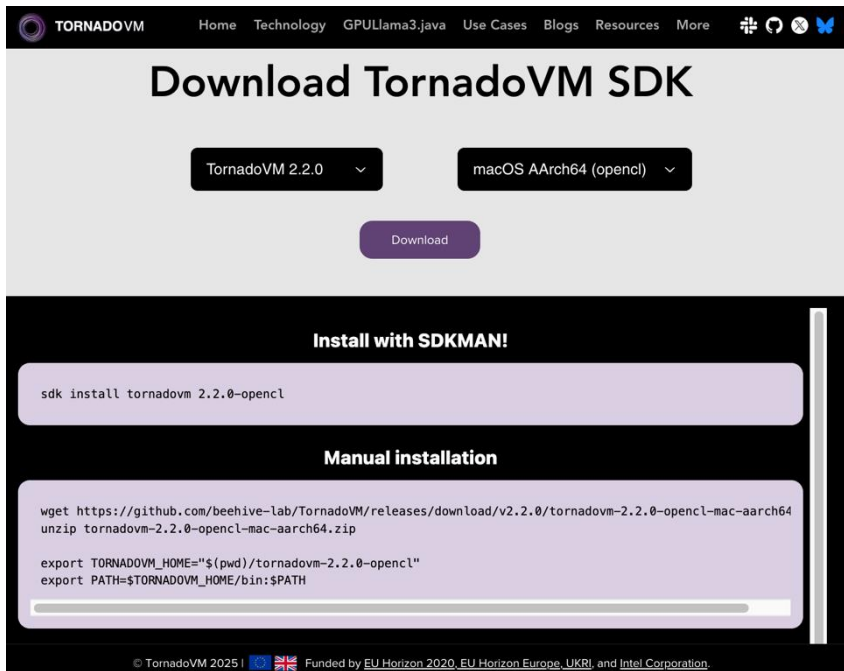
Factory methods to
migrate your **data** to
off-heap data types!

Factory method to
migrate off-heap **data** to
on-heap data types!

<https://tornadovm.readthedocs.io/en/latest/offheap-types.html#migrating-from-v0-15-2-to-v1-0>

Setup TornadoVM

SDK archives are shipping!



The screenshot shows the TornadoVM website's download page. At the top, there's a navigation bar with links like Home, Technology, GPU/Llama3.java, Use Cases, Blogs, Resources, and More. The main heading is "Download TornadoVM SDK". Below this, there are two dropdown menus: "TornadoVM 2.2.0" and "macOS AArch64 (openc1)". A purple "Download" button is positioned below these. Further down, under the heading "Install with SDKMAN!", there is a code block with the command `sdk install tornadovm 2.2.0-openc1`. Below that, under "Manual installation", there is a code block with commands to wget, unzip, and set environment variables for TORNADOVM_HOME and PATH. At the bottom, there is a footer with copyright information and funding sources.

TORNADOVM Home Technology GPU/Llama3.java Use Cases Blogs Resources More

Download TornadoVM SDK

TornadoVM 2.2.0 macOS AArch64 (openc1)

Download

Install with SDKMAN!

```
sdk install tornadovm 2.2.0-openc1
```

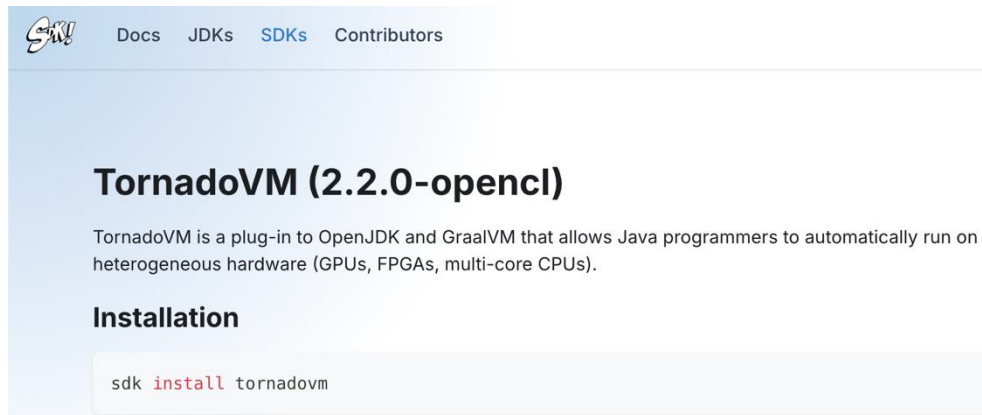
Manual installation

```
wget https://github.com/bee-hive-lab/TornadoVM/releases/download/v2.2.0/tornadovm-2.2.0-openc1-mac-aarch64.zip
unzip tornadovm-2.2.0-openc1-mac-aarch64.zip

export TORNADOVM_HOME="$(pwd)/tornadovm-2.2.0-openc1"
export PATH=$TORNADOVM_HOME/bin:$PATH
```

© TornadoVM 2025! Funded by EU Horizon 2020, EU Horizon Europe, UKRI, and Intel Corporation.

<https://www.tornadovm.org/downloads>



The screenshot shows the SDKMAN! website. The navigation bar includes links for Docs, JDKs, SDKs, and Contributors. The main heading is "TornadoVM (2.2.0-openc1)". Below this, there is a paragraph describing TornadoVM as a plug-in to OpenJDK and GraalVM that allows Java programmers to automatically run on heterogeneous hardware (GPUs, FPGAs, multi-core CPUs). Under the heading "Installation", there is a code block with the command `sdk install tornadovm`.

SDKMAN! Docs JDKs SDKs Contributors

TornadoVM (2.2.0-openc1)

TornadoVM is a plug-in to OpenJDK and GraalVM that allows Java programmers to automatically run on heterogeneous hardware (GPUs, FPGAs, multi-core CPUs).

Installation

```
sdk install tornadovm
```



<https://sdkman.io/sdks/tornadovm/>

How to Install TornadoVM?



- Ensure you have a valid JDK distribution (e.g. Temurin):

```
sdk install java 21.0.9-tem
```

- Install TornadoVM SDK:

```
sdk install tornadovm 2.2.0-openc1
```

→ \$JAVA_HOME

```
tornado --devices
```

→ \$TORNADOVM_HOME

```
tornado --version
```

What is the tornado command?

```
$ tornado -cp MyApp.jar MyApp
```

```
$ java @$TORNADOVM_HOME/tornado-argfile -cp MyApp.jar MyApp
```


What is the tornado command?

```
$ tornado -cp MyApp.jar MyApp
```

```
$ java @$TORNADOVM_HOME/tornado-argfile -cp MyApp.jar MyApp
```

What is the tornado command?

```
$ tornado -cp MyApp.jar MyApp
```

```
$ java @$TORNADOVM_HOME/tornado-argfile -cp MyApp.jar MyApp
```



Contains the **JVM Flags** that enable TornadoVM:

tornado --printJavaFlags

```
-server
-XX:+UnlockExperimentalVMOptions
-XX:+EnableJVMCI
--enable-preview
-Djava.library.path=/Users/thanos/.sdkman/candidates/tornadovm/current/lib
--module-path ./Users/thanos/.sdkman/candidates/tornadovm/current/share/java/tornado
-Dtornado.load.api.implementation=uk.ac.manchester.tornado.runtime.tasks.TornadoTaskGraph
-Dtornado.load.runtime.implementation=uk.ac.manchester.tornado.runtime.TornadoCoreRuntime
-Dtornado.load.tornado.implementation=uk.ac.manchester.tornado.runtime.common.Tornado
-Dtornado.load.annotation.implementation=uk.ac.manchester.tornado.annotation.ASMClassVisitor
-Dtornado.load.annotation.parallel=uk.ac.manchester.tornado.api.annotations.Parallel --upgrade-module-path
/Users/thanos/.sdkman/candidates/tornadovm/current/share/java/graalJars
-XX:+UseParallelGC @/Users/thanos/.sdkman/candidates/tornadovm/current/etc/exportLists/common-exports
@/Users/thanos/.sdkman/candidates/tornadovm/current/etc/exportLists/opencv-exports --add-modules ALL-
SYSTEM,tornado.runtime,tornado.annotation,tornado.drivers.common,tornado.drivers.opencv
```

What is the tornado command?

```
$ tornado -cp MyApp.jar MyApp
```

```
$ java @$TORNADOVM_HOME/tornado-argfile -cp MyApp.jar MyApp
```



Contains the **JVM Flags** that enable TornadoVM:

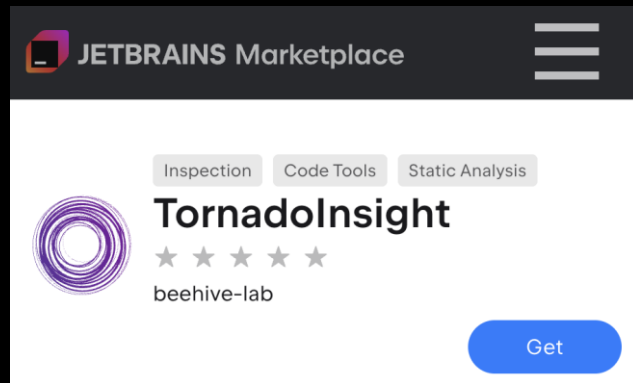
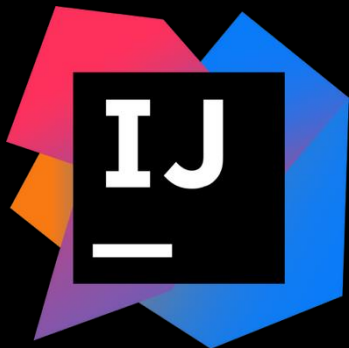
```
tornado --printJavaFlags
```

```
$ java @$TORNADOVM_HOME/tornado-argfile -cp $TORNADOVM_HOME/share/java/tornado/tornado-  
examples-2.2.0.jar uk.ac.manchester.tornado.examples.compute.MatrixVectorRowMajor
```

A TornadoVM Example of MatrixMultiplication

<https://github.com/beehive-lab/TornadoVM>

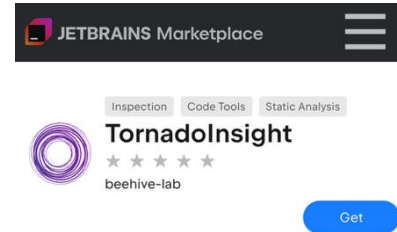
Branch: *tornado-insight/demos*



IntelliJ & Plugin configuration

- Ensure you have installed **JetBrains IntelliJ IDEA** & **TornadoInsight (1.5.0)**

<https://plugins.jetbrains.com/plugin/23309-tornadoinsight>



<https://p2code-project.eu/>



UK Research
and Innovation

Funded by the European Union (P2CODE: 101093069) & UKRI under the Horizon Europe funding guarantee (P2CODE: 10048316).

IntelliJ & Plugin configuration

- Ensure

<https://pluc>

 **TORNADOVM**

Home Technology GPULlama3.java Use Cases Blogs Resources More 

 Thanos Stratikopoulos · Dec 23, 2025 · 2 min read 

TornadoInsight - Compatibility with TornadoVM SDK 2.0+ & Configuration Guide



This blog updates the previously published **TornadoInsight** configuration [guidelines](#) and explains how to configure the required environment variable to ensure that TornadoInsight correctly detects the TornadoVM SDK when IntelliJ IDEA is launched from a graphical environment.

Overview

TornadoInsight requires access to a compatible Java Development Kit (JDK) and the TornadoVM SDK. We explained in previous blogs the guidelines for installing the TornadoVM SDKs ([manually](#)), or via [SDKMAN!](#).

<https://tornadovm.org/blog>

TornadoVM Deep-dive



DevoxxBE 2025

DEVOXX™

Cloud ORACLE

- AI-ready
- Production LLMs
- Native GPU execution
- Integration with Java tooling ecosystem
- Support for AI Java frameworks
- Production deployments
- Integrated with Langchain4J

[Play Video](#)

GPUllama3 java: Beyond CPU Inference with Modern Java

The modern inference of LLaMA3 Java on GPU is a game-changer for AI applications. It allows for faster inference times, reducing latency and improving user experience. This is achieved by leveraging the power of GPUs, which are designed for parallel processing. The slide also includes a performance comparison graph showing the difference between CPU and GPU inference times.

Google Cloud ING ORACLE AWS JETBRAINS

Integration in Java Projects

How to use TornadoVM in my project?



- Jars pushed in Maven Central



Integrating TornadoVM in your project

For Maven projects (pom.xml)

```
<dependencies>
  <dependency>
    <groupId>io.github.beehive-lab</groupId>
    <artifactId>tornado-api</artifactId>
    <version>2.2.0</version>
  </dependency>
  <dependency>
    <groupId>io.github.beehive-lab</groupId>
    <artifactId>tornado-runtime</artifactId>
    <version>2.2.0</version>
  </dependency>
</dependencies>
```



More Complex Cases (GPULlama3.java)

GPULLama3.java



GitHub repository page for **GPULLama3.java** by **beehive-lab**.

Navigation: Code | Issues 15 | Pull requests 6 | Discussions | Actions | Projects | Wiki | Security | Insights | Settings

Repository details: **GPULLama3.java** (Public) | Edit Pins | Unwatch 10 | Fork 26 | Star 225

Branches: main | 24 Branches | 8 Tags | Go to file | Code

Recent activity by **mikepapadim** (Merge pull request #95 from beeh...):

File	Commit Message	Time
.github	Refine tag pattern in deploy-maven-cent...	last month
.mvn/wrapper	Add Maven wrapper support	3 months ago
docs	[CI][docs]Add release automation docu...	last month
scripts	Move helper scripts under scripts	3 months ago
src/main/java/org/beehive/gpulla...	Rename Granite8_0LayerPlanner to Gran...	last month

About: GPU-accelerated Llama3.java inferenc in pure Java using TornadoVM.

Link: github.com/beehive-lab/GPULLama3.java

Tags: java, gpu, nvidia, compilers, granite, mistral, accelerators, tornadovm, llm, java21, gguf, mistral-7b, llama3, phi-3, phi-3-min, ibm-granite, qwen2-5, deepseek-r1, qwen3

<https://github.com/beehive-lab/GPULLama3.java>

About GPULLama3.java

- Java-based GPU capable LLM Inference Engine
 - built on top of llama3.java for model ingestion
 - built with **TornadoVM** for **GPU acceleration**
- Allows switching between pure-CPU to pure-GPU inference
- Supports GGUF (General Graph Universal Format) model format
- Supported Models:
 - Llama3
 - Mistral
 - Phi-3
 - Qwen2.5, 3
 - **IBM Granite 3.3, 4.0 (new)**
 - Gemma (soon)
- Integration with LangChain4j & Quarkus

Integration with Langchain4j & Quarkus



Integrating GPULLama3.java with LangChain4j in your project

For Maven projects (pom.xml)

```
<dependency>
  <groupId>dev.langchain4j</groupId>
  <artifactId>langchain4j</artifactId>
  <version>1.9.1</version>
</dependency>

<dependency>
  <groupId>dev.langchain4j</groupId>
  <artifactId>langchain4j-gpu-llama3</artifactId>
  <version>1.9.1-beta17</version>
</dependency>
```

For Gradle projects (build.gradle)

```
implementation 'dev.langchain4j:langchain4j:1.9.1'
implementation 'dev.langchain4j:langchain4j-gpu-llama3:1
```



Integrating GPULLama3.java with Quarkus in your project

For Maven projects (pom.xml)

```
<dependency>
  <groupId>io.quarkiverse.langchain4j</groupId>
  <artifactId>quarkus-langchain4j-gpu-llama3</artifactId>
  <version>1.4.2</version>
</dependency>
```



Integrating GPULLama3.java in your project

For Maven projects (pom.xml)

```
<dependency>
  <groupId>io.github.beehive-lab</groupId>
  <artifactId>gpu-llama3</artifactId>
  <version>0.3.3</version>
</dependency>
```



Demo GPULLama3.java, Quarkus, LangChain4j

The screenshot shows the Quarkiverse documentation website. The top navigation bar includes the Quarkiverse logo, 'DOCS', and links for 'LEARN', 'EXTENSIONS', and 'COMMUNITY'. The breadcrumb trail indicates the current page is 'LangChain4j / Getting Started / Implementing a summarization service'. A search bar and 'Edit this Page' link are also present. The left sidebar lists the 'LangChain4j' section with sub-items: 'Getting Started' (which is expanded to show 'Quick start', 'Implementing a summarization service', 'Extracting text from an image', 'Implementing a simple RAG application', and 'Using function calling'), 'Guides', 'Reference', and 'Development'. The main content area features the title 'Implementing a Summarization Service' and a brief introduction: 'This guide shows you how to implement a simple text summarization service using Quarkus LangChain4j and run it as a standalone CLI application. The service reads a plain text file (such as an article or report), sends the content to a Large Language Model, and receives a structured Markdown summary in return.' Below this is a 'Project Setup' section with the text: 'Make sure you've added your preferred LLM provider to your pom.xml. For example, to use OpenAI:'. A code block for a dependency is shown with the placeholder '<dependency>'. A 'Contents' sidebar on the right lists: 'Project Setup', 'Defining the Summarization Service', 'Writing the Main Application', and 'Running the Application'.

QUARKIVERSE DOCS

LEARN ▾ EXTENSIONS ▾ COMMUNITY ▾

LangChain4j / Getting Started / Implementing a summarization service

Search by keyword Edit this Page

LangChain4j

- ▾ Getting Started
 - Quick start
 - Implementing a summarization service
 - Extracting text from an image
 - Implementing a simple RAG application
 - Using function calling
- Guides
- Reference
- Development

Implementing a Summarization Service

This guide shows you how to implement a simple text summarization service using Quarkus LangChain4j and run it as a standalone CLI application. The service reads a plain text file (such as an article or report), sends the content to a Large Language Model, and receives a structured Markdown summary in return.

Project Setup

Make sure you've added your preferred LLM provider to your `pom.xml`. For example, to use OpenAI:

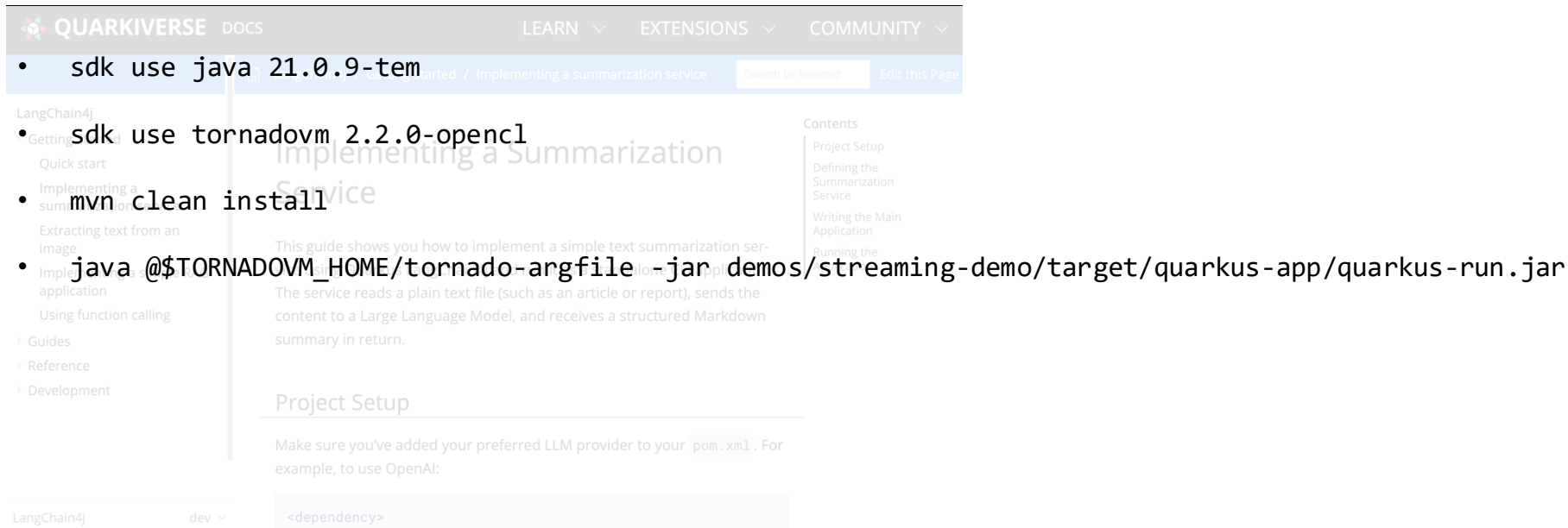
```
<dependency>
```

Contents

- Project Setup
- Defining the Summarization Service
- Writing the Main Application
- Running the Application

<https://docs.quarkiverse.io/quarkus-langchain4j/dev/quickstart-summarization.html>

Demo GPULLama3.java Quarkus, LangChain4j



The screenshot shows the Quarkiverse documentation website. The top navigation bar includes 'QUARKIVERSE', 'DOCS', 'LEARN', 'EXTENSIONS', and 'COMMUNITY'. The main content area is titled 'Implementing a Summarization Service' and includes a 'Project Setup' section. The left sidebar lists various guides, and the right sidebar shows a table of contents. The main text describes how to implement a simple text summarization service using a Large Language Model (LLM) and provides a code snippet for the project setup.

- `sdk use java 21.0.9-tem`
- `sdk use tornadovm 2.2.0-openc1`
- `mvn clean install`
- `java @$TORNADOVM_HOME/tornado-argfile -jar demos/streaming-demo/target/quarkus-app/quarkus-run.jar`

Project Setup

Make sure you've added your preferred LLM provider to your `pom.xml`. For example, to use OpenAI:

```
<dependency>
```

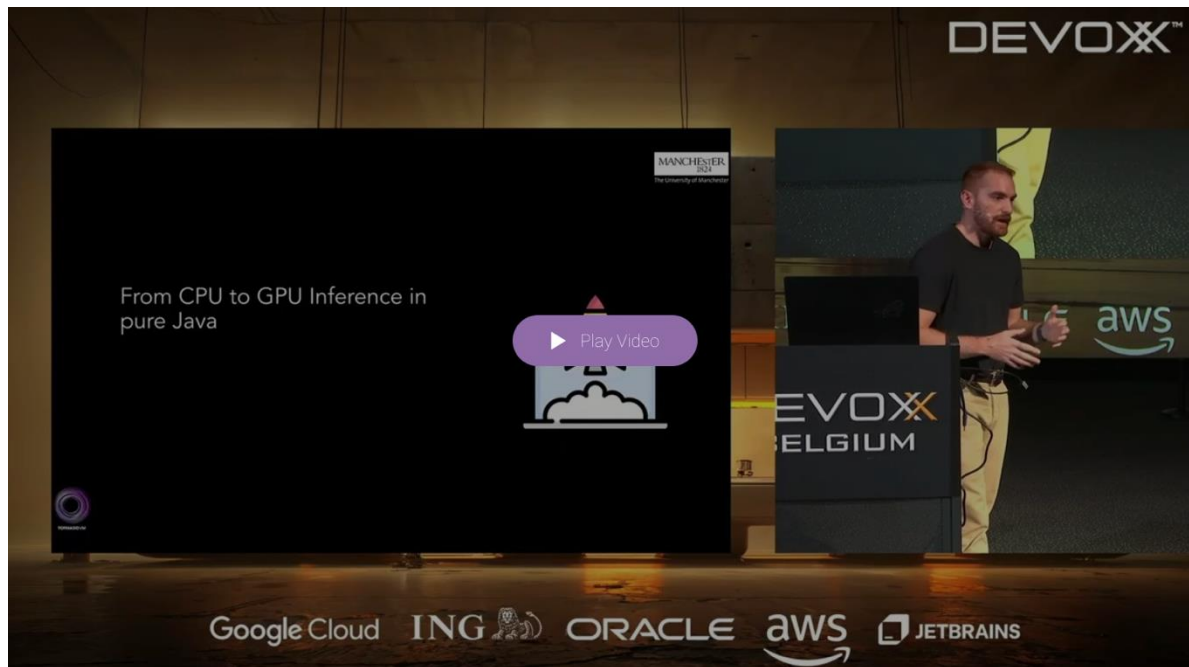
<https://docs.quarkiverse.io/quarkus-langchain4j/dev/quickstart-summarization.html>

<https://github.com/beehive-lab/gpullama3-quarkus-langchain4j-demo>

More about it (Devoxx Belgium 2025)



DevovxBE 2025



Summary

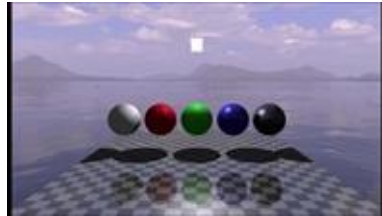
Use Cases

<https://www.tornadovm.org/use-cases>



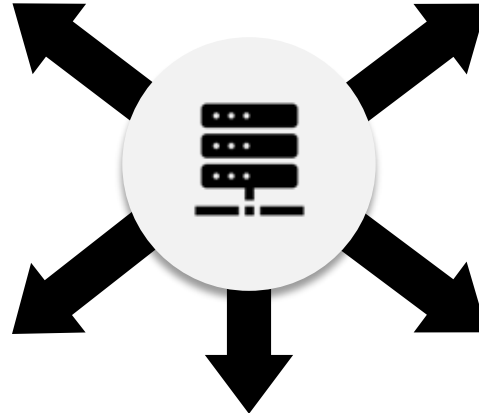
COMPUTER VISION

47x Performance



RAY TRACING

200x Performance



FACE DETECTION

23x Performance



MACHINE LEARNING

14x Performance



DATA ANALYTICS
& NLP

Levenshtein Distance	9.8x
K-means Clustering	9.7x
Hierarchical Clustering	28x

Future Roadmap

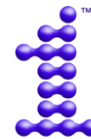
- Performance improvements across GPU vendors
- Extended platform support (Metal, AMD)
- AI-specific optimizations and platform integrations
- Compatibility with JDK 25
- Continue successful integration of future Java features
 - Project Babylon, Valhalla, Leyden

Summary

- TornadoVM is an open-source plugin for JDK vendors
- Enables native GPU acceleration of Java programs
- AI-ready (GPULLama3.java, LangChain4J, etc.)
- Production ready
- Rich tool ecosystem
(Development, Deployment, Performance Monitoring)



encrypt



oneAPI



UK Research
and Innovation

Our Team

Academic Staff

- Christos Kotselidis

Research Staff

- Orion Papadakis
- Michail Papadimitriou
- Maria Xekalaki
- Thanos Stratikopoulos
- Ruiqi Ye

Alumni

- | | |
|------------------|---------------------------|
| • James Clarkson | • Florin Blanaru |
| • Foivos Zakkak | • Gyorgy Rethy |
| • Juan Fumero | • Mihai-Christian Olteanu |
| • Benjamin Bell | • Ian Vaughan |
| • Amad Aslam | • Tianyu Zuo |
| • Ales Kubicek | |



Questions?



<https://www.linkedin.com/company/tornadovm>



@tornadovm



@tornadovm.org



info@tornadovm.org



<https://tornadovm.org>



<https://github.com/beehive-lab/TornadoVM>

<https://github.com/beehive-lab/GPULLama3.java>

Demo 1 <https://github.com/beehive-lab/GPULLama3.java>

Demo 2 <https://github.com/beehive-lab/gpullama3-quarkus-langchain4j-demo>



encrypt



oneAPI



UK Research
and Innovation